ITA14 – ETHICAL HACKING

ASSIGNMENT 3 – CO3

PART B – STUDENT B ANSWERS ONLY

Scan-Data Validation, Standardization, Integration, File Handling and Reusable Pipeline Design

Student Name: Vempati Murale Karrtik

Register Number: 192312567

Department: _____

Pair / Group: Harishwar Reddy

Submission Date: _____

Scope note: This document answers only the responsibilities assigned to Student B in Q1–Q7. Student A-only answers and the original question paper are intentionally omitted. Shared design requirements are included only where needed to explain Student B's work.

PART B – SCOPE AND WORKING ASSUMPTIONS

This submission follows the Student B responsibilities in the uploaded Assignment 3 question paper. It focuses on service relationships and enquiry design; validation and standardization; summary casting; OS/version and enumeration integration; export and read-back validation; and the Student B portions of the reusable Bash function library.

Authorized-lab boundary

All examples are designed for systems owned by the university or explicitly authorized for assessment. Network discovery, service detection and enumeration commands must be run only against the approved lab subnet and named test devices. No command in this document authorizes scanning public or third-party systems.

Illustrative lab inventory

Asset	Role	Example address / identifier	Scope status
LAB-WIN-01	Windows test VM	192.168.56.20	In scope
LAB-AND-01	Android test device / emulator	192.168.56.30	In scope
LAB-JAVA-01	Java RMI test VM	192.168.56.40	In scope
LAB-NW-01	NetWare-compatible training target	192.168.56.50	In scope if available
LAB-BT-01	Bluetooth training device	Recorded device ID	In scope if specifically approved
Campus production portal	Production service	Not listed in lab inventory	Out of scope

Where a platform or tool is unavailable, the dataset must record “not tested” or “not available” rather than inventing a result. The sample rows and outputs below are illustrative test data, not claims that a real network was scanned.

Q1 – STUDENT B: SERVICE RELATIONSHIP AND ENQUIRY DESIGN

Student B leads the relationship model for Services, OS/Versions and Enumeration findings. These datasets describe what a host appears to run, which service is associated with a port, and what platform-specific enumeration reports. The common parent key is the host IP address. Service-level records additionally require port and protocol.

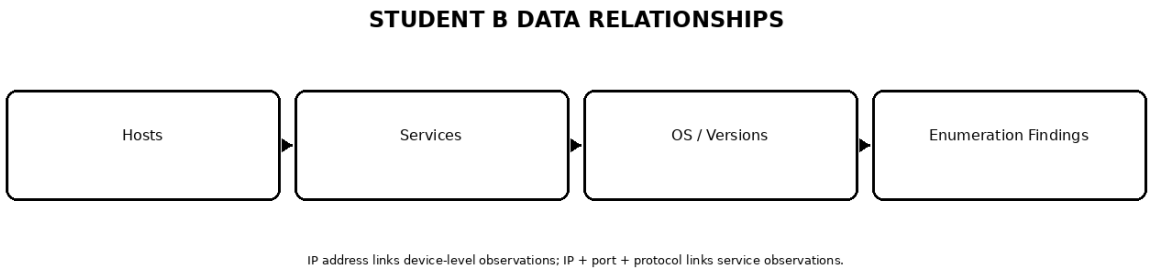


Figure 1. Relationship model for Student B datasets.

Dataset	Purpose	Primary / candidate key	Foreign key
Hosts	One row per discovered host	host_id or ip_address	—
Services	One row per observed endpoint	ip_address + port + protocol	ip_address → Hosts
OS_Versions	OS guess and version observations	ip_address + observation_id	ip_address → Hosts
Enumeration	Platform-specific evidence	ip_address + source + finding_id	ip_address → Hosts

The model is intentionally one-to-many: one host can expose multiple services and can have multiple OS or enumeration observations. The design does not force a single OS guess or a single enumeration result into one host row.

Q1 – SERVICE, OS/VERSION AND ENUMERATION SCHEMAS

CSV file	Column headers	Data types / constraints
services.csv	ip_address, port, protocol, service_name, product, version, state, scan_id, observed_at	IPv4/IPv6 text; port integer 1–65535; protocol tcp/udp; state enum; timestamps ISO-8601
os_versions.csv	ip_address, os_family, os_name, os_version, accuracy, method, scan_id, observed_at	IP text; accuracy integer 0–100; method text; nullable version
enumeration.csv	ip_address, platform, source_tool, category, finding_key, finding_value, confidence, scan_id, observed_at	IP text; category controlled vocabulary; confidence 0–100; finding value text

service_aliases.csv	alias, canonical_name, protocol, notes	Normalized lowercase strings; unique alias + protocol

Why these fields are necessary

- ip_address is the stable link from every observation to the host inventory.
- port and protocol distinguish TCP/80 from UDP/80; the port number alone is not a unique service key.
- service_name is standardized for analysis, while product and version preserve the scanner's original detail.
- source_tool and method provide provenance: the reader can identify how an observation was obtained.
- scan_id and observed_at separate observations from different sessions and make comparisons reproducible.

Example CSV headers

ip_address,port,protocol,service_name,product,version,state,scan_id,observed_at

ip_address,os_family,os_name,os_version,accuracy,method,scan_id,observed_at

ip_address,platform,source_tool,category,finding_key,finding_value,confidence,scan_id,observed_at

Q1 – PLATFORM-SPECIFIC ENUMERATION RELATIONSHIPS

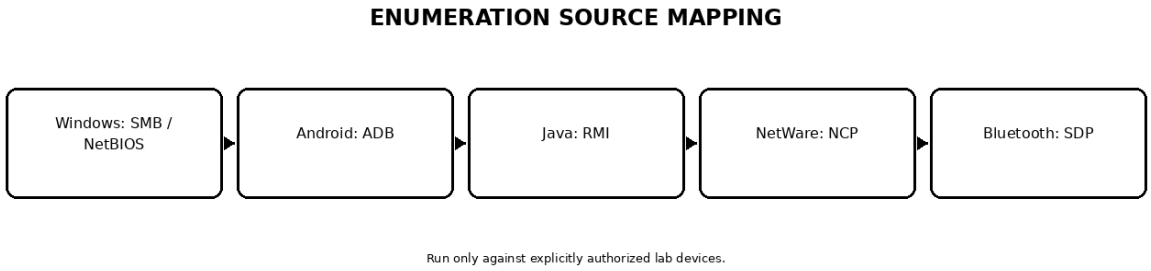


Figure 2. Enumeration sources are platform-specific and must retain source attribution.

Platform	Possible source / tool	Example observation fields	Linking key
Windows	enum4linux / nbtscan	NetBIOS name, workgroup, SMB information	ip_address + scan_id
Android	ADB on authorized device	Build fingerprint, release, SDK level, device model	ip_address or device_id + scan_id
Java	RMI registry inspection tool	Registry endpoint, bound service names, port	ip_address + port + protocol
NetWare	NCP server information tool	Server name, NCP service details, version if exposed	ip_address + scan_id
Symbian-type Bluetooth	SDP browse on approved device	Service UUID, service name, device address	device_id + scan_id

The source question paper names Windows, Symbian, Java OS, Android and NetWare. The schema therefore keeps platform as a controlled field rather than assuming every host is a conventional Windows or Linux server.

Relationship rules

- Every enumeration row must refer to a known host or explicitly be placed in an unmatched-observation queue.
- A platform observation must include the tool or method and collection timestamp.
- Sensitive values such as usernames, device identifiers or configuration details must be minimized and redacted when they are not required.
- A scanner guess is an observation, not definitive proof of the operating system.

Q1 – TESTABLE ENQUIRY QUESTIONS

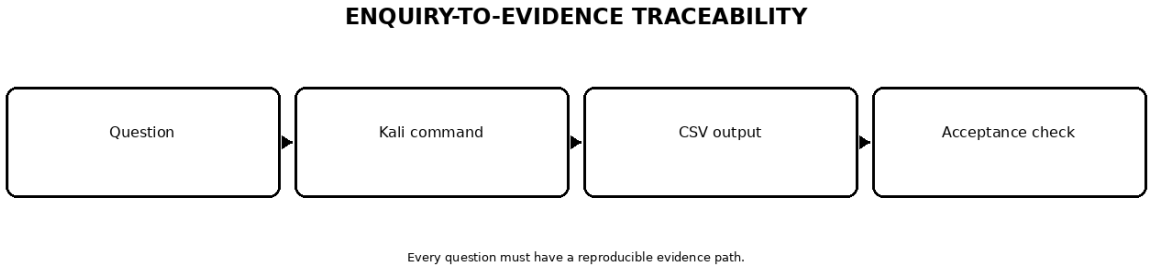


Figure 3. Each enquiry question is connected to a command, an output dataset and an acceptance check.

Enquiry question	Authorized command / pipeline	Output / acceptance criterion
Which approved hosts expose an SMB-related service?	nmap -sV -p 139,445 192.168.56.0/24 -oX smb.xml	CSV rows contain valid IP, port, protocol and service; all IPs belong to scope.
Which service versions were observed on approved hosts?	nmap -sV -p 22,80,443,445 192.168.56.0/24 -oG services.gnmap	Service/version records preserve host, port and protocol; unknown version remains blank, not guessed.
Which Windows lab hosts return NetBIOS information?	nbtscan 192.168.56.0/24	Every result maps to an approved IP and includes source_tool=nbtscan.
Which OS observations have low confidence?	awk -F, 'NR>1 && $5<70 {print}' os_versions.csv	All confidence values are numeric 0–100; low-confidence rows are flagged for review.
Which enumeration records have no matching host?	join -t, -v 1 <(sort enumeration.csv) <(sort hosts.csv)	Unmatched rows are exported to a review file and not discarded.

Acceptance criteria

- Correctness: values follow the schema and allowed ranges.
- Consistency: service names, protocol labels and platform labels use standard forms.
- Traceability: each row has scan_id, source_tool or method, and observed_at.
- Reproducibility: identical source files and transformation version produce identical normalized output.

Q2 – STUDENT B: VALIDATION AND STANDARDIZATION

Student B constructs Services, OS/Versions and Enumeration CSV datasets and implements validation before merging. The validation sequence starts with file integrity and schema checks because later checks depend on knowing which column contains each value.

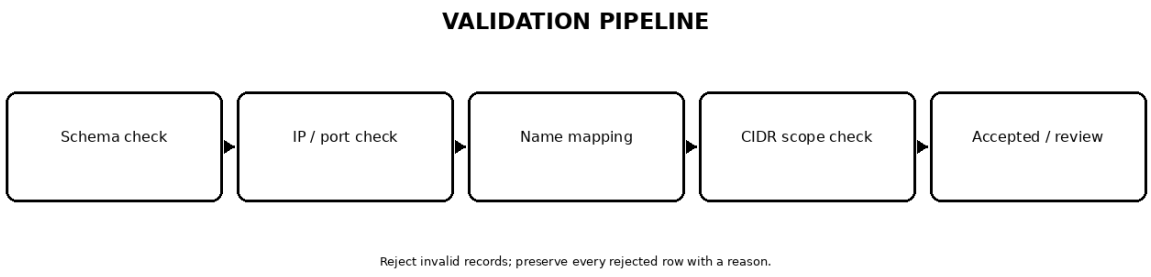


Figure 4. Validation stages. Records that fail are preserved with a rejection reason.

Validation order and rationale

Order	Check	Why first / expected action
1	File exists, readable, non-empty	Prevents processing a missing or empty input.
2	Header and column count	Prevents values being interpreted under the wrong field.
3	Required values	Flags missing IP, port, protocol, scan_id or source.
4	IP and port validity	Rejects malformed addresses and invalid numeric ranges.
5	Protocol and service normalization	Ensures consistent joins and summaries.
6	Scope membership	Separates out-of-scope observations before

		integration.
7	Duplicate and relationship checks	Detects repeated endpoints and orphaned host references.

The first failed validation stage is recorded for each row. Multiple issues may be collected where practical, but the pipeline must not silently convert an invalid value into a plausible value.

Q2 – CONSTRUCTING SERVICES, OS/VERSIONS AND ENUMERATION DATA

Sample services.csv

```
ip_address,port,protocol,service_name,product,version,state,scan_id,observed_at
192.168.56.20,445,tcp,microsoft-ds,Samba,4.15,open,S001,2026-11-04T10:00:00Z
192.168.56.20,139,tcp,netbios-ssn,,unknown,open,S001,2026-11-04T10:00:00Z
192.168.56.40,1099,tcp,java-rmi,Java RMI,,open,S001,2026-11-04T10:05:00Z
```

Sample os_versions.csv

```
ip_address,os_family,os_name,os_version,accuracy,method,scan_id,observed_at
192.168.56.20,Windows,Windows Server,2022,92,nmap-O,S001,2026-11-04T10:02:00Z
192.168.56.30,Android,Android,,68,adb-getprop,S001,2026-11-04T10:03:00Z
```

Sample enumeration.csv

```
ip_address,platform,source_tool,category,finding_key,finding_value,confidence,scan_id,observed_at
192.168.56.20,Windows,nbtscan,netbios,name,LAB-WIN-01,95,S001,2026-11-04T10:04:00Z
192.168.56.30,Android,adb,device,ro.build.version.release,14,100,S001,2026-11-04T10:05:00Z
```

These rows are synthetic examples to demonstrate structure only. The actual submission should replace them with the group's authorized lab output and retain the original scanner files as evidence.

Q2 – VALIDATION COMMANDS AND CHECKS

Required-column check

```
awk -F, 'NR==1 { if ($1!="ip_address" || $2!="port" || $3!="protocol") exit 1 }' services.csv
```

A production-grade CSV parser should be used if fields may contain quoted commas. The simple awk examples assume the controlled sample CSV format has no embedded commas.

IP pattern and port-range checks

```
awk -F, 'NR>1 && $1 !~ /^([0-9]{1,3}\.){3}[0-9]{1,3}$/ {print "BAD_IP:",$0}' services.csv
awk -F, 'NR>1 && ($2 !~ /^[0-9]+$/ || $2<1 || $2>65535) {print "BAD_PORT:",$0}' services.csv
```

The regular expression checks IPv4 shape only. A second octet-range check is required because a pattern can still accept values such as 999.999.1.1.

```
awk -F. 'NF==4 {ok=1; for(i=1;i<=4;i++) if($i !~ /^[0-9]+$/ || $i<0 || $i>255) ok=0; if(!ok) print "INVALID_IPV4" }'
```

Protocol and service-name checks

```
awk -F, 'NR>1 && $3!="tcp" && $3!="udp" {print "BAD_PROTOCOL:",$0}' services.csv
grep -Eiv ',((tcp)|(udp)),' services.csv
```

The grep expression is a quick diagnostic, not a replacement for a full validator. Validation should return explicit accepted and rejected datasets with reason codes.

Q2 – SERVICE-NAME LOOKUP AND CIDR SCOPE VALIDATION

Canonical service-name lookup

Alias	Canonical name	Protocol	Reason
www	http	tcp	Common shorthand
http-alt	http-alt	tcp	Distinct alternate HTTP service
HTTP	http	tcp	Case normalization
https	https	tcp	Canonical HTTPS label
microsoft-ds	microsoft-ds	tcp	SMB-related service label

netbios-ssn	netbios-ssn	tcp	NetBIOS session service

```
awk -F, 'BEGIN{while((getline<"service_aliases.csv")>0) map[tolower($1)]=$2} NR==1{print;next} { $4=tolower($4); if($4 in map) $4=map[$4]; print }' OFS=, services.csv
```

CIDR scope validation

The approved assessment subnet in this example is 192.168.56.0/24. grepcidr can test whether an IP address belongs to the approved range. The allowlist must be supplied by the engagement owner; never infer authorization from network reachability.

```
cut -d, -f1 services.csv | tail -n +2 | grepcidr 192.168.56.0/24
cut -d, -f1 services.csv | tail -n +2 | grepcidr -v 192.168.56.0/24 > out_of_scope_ips.txt
```

Out-of-scope rows are quarantined and escalated. They are not merged into the approved assessment dataset.

Q2 – VALIDATION RESULTS AND LARGE-ASSESSMENT PRIORITIES

Dataset	Input rows	Accepted	Review	Rejected	Reason example
Services	120	112	5	3	Invalid port / out-of-scope IP
OS_Versions	20	16	4	0	Low-confidence OS guess
Enumeration	35	31	3	1	Missing source attribution

Illustrative reconciliation: for Services, 120 input records = 112 accepted + 5 review + 3 rejected. Every rejected row is retained in a separate file with a reason code. Counts must be calculated from the actual input files during the lab.

Which checks should be applied first?

1. File and schema integrity: stop immediately if the file is missing, unreadable or has unexpected columns.
2. Required-field presence: ensure keys and provenance are available before any join.
3. Type and range validation: validate IP addresses, port numbers, protocol values and confidence scores.
4. Scope membership: prevent unauthorized or unrelated observations from entering the approved dataset.
5. Normalization and duplicates: standardize service names, then detect duplicate endpoint observations.
6. Cross-dataset referential checks: identify enumeration or version rows without a matching host.

This order minimizes wasted work and prevents invalid records from contaminating joins. Scope checks occur early enough to isolate out-of-scope data before it can enter reports.

Q3 – STUDENT B: SERVICE-NAME STANDARDIZATION AND TEST VALIDATION

Student B owns the standardization rules for inconsistent service names and the verification tests that prove the rules behave as intended. Standardization improves grouping and comparison, but it must preserve the original scanner label for auditability.

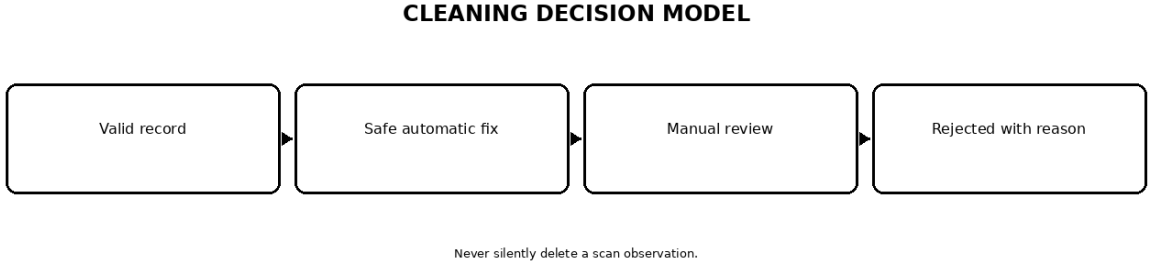


Figure 5. Cleaning decisions separate safe normalization from manual review.

Raw value	Normalized value	Decision	Reason
HTTP	http	Automatic	Case normalization is unambiguous.

www	http	Automatic with mapping	Approved alias mapping.
http-alt	http-alt	Keep distinct	Alternate service must not be collapsed into http.
unknown	unknown	Manual review	Scanner did not identify service.
blank	blank / missing	Manual review	No evidence to infer a service.
ftp-data	ftp-data	Keep distinct	Different service label.

Lookup file design

```
alias,canonical_name,protocol,notes
HTTP,http,tcp,case-normalized HTTP
www,http,tcp,approved common alias
http-alt,http-alt,tcp,retain alternate service
https,https,tcp,canonical HTTPS label
```

The mapping file is version-controlled. Each change includes an author, date and reason. A mapping is applied only when the alias and protocol combination are known.

Q3 – REUSABLE STANDARDIZATION OPERATIONS

Normalize case and whitespace

```
awk -F, 'BEGIN{OFS=","} NR==1{print;next} {gsub(/^[ \t]+|[ \t]+$/, "", $4); $4=tolower($4); print}' services.csv
```

Apply an approved alias map

```
awk -F, 'BEGIN{OFS=","; while((getline<"service_aliases.csv")>0) map[tolower($1)]=$2} NR==1{print;next} {k=tolower($4); if(k in map) $4=map[k]; print}' services.csv
```

Flag unknown or blank names

```
awk -F, 'NR>1 && ($4=="" || $4=="unknown") {print "REVIEW:",$0}' services_normalized.csv
```

Detect duplicate endpoint observations

```
tail -n +2 services_normalized.csv | sort -t, -k1,1 -k2,2n -k3,3 -k8,8 | uniq -d
```

Duplicate detection must use the intended observation key. For a single scan session, IP + port + protocol + scan_id may identify a duplicate. Across different scan sessions, the same endpoint is not necessarily a duplicate; it may be a valid repeated observation.

Q3 – SERVICE STANDARDIZATION TEST MATRIX

Test ID	Input	Expected output	Pass condition
SVC-01	HTTP	http	Canonical label emitted
SVC-02	www	http	Alias map applied
SVC-03	http-alt	http-alt	Alternate label retained
SVC-04	HTTPS	https	Case normalized
SVC-05	unknown	Manual review	Not silently replaced
SVC-06	empty field	Manual review	Missing value flagged
SVC-07	same endpoint twice in S001	One accepted + duplicate audit row	Duplicate is traceable
SVC-08	same endpoint in S001 and S002	Both retained	Session identity preserved

Boundary and invalid cases

- Port 0: reject as invalid for a TCP/UDP service endpoint in this dataset.
- Port 1 and 65535: accept if the source record and protocol are valid.
- Port 65536: reject because it exceeds the valid port range.
- IP 192.168.56.255: syntactically valid IPv4 address but may be the subnet broadcast address; flag according to network policy.
- IP 192.168.56.256: reject because an IPv4 octet cannot exceed 255.

Q3 – AUDIT TRAIL AND JOINT CONCLUSION SUPPORT

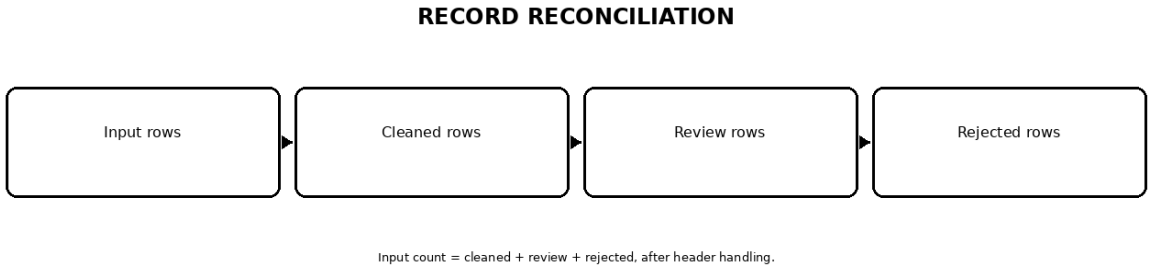


Figure 6. A cleaning run must reconcile every input record.

Record status	Meaning	Required evidence
Accepted	Valid and standardized record	Cleaned CSV row
Review	Ambiguous or low-confidence record	Review CSV + reason
Rejected	Invalid or out-of-scope record	Rejected CSV + reason
Duplicate	Repeated observation under defined key	Duplicate audit file

Record-count reconciliation

```
wc -l services.csv services_clean.csv services_review.csv services_rejected.csv
diff -u <(sort services_expected.csv) <(sort services_clean.csv)
```

The row-count equation is applied after excluding the header: input data rows = accepted + review + rejected. Duplicate rows should be assigned to one of these dispositions and separately tagged, so that the same row is not counted twice.

Why automated cleaning is not enough

Automated cleaning cannot determine whether a scanner's observation is factually correct, whether a service is intentionally exposed, or whether an operating-system fingerprint is reliable. A syntactically valid record may still be misleading. For example, a service may be identified by a port-number heuristic even when it is running on a non-standard port, and an OS guess may have low confidence because of filtering or network middleboxes.

Human review is required for ambiguous service aliases, low-confidence fingerprints, conflicting results between tools and any record that suggests an unexpected sensitive service. Cleaning improves consistency; it does not replace analyst judgement or validation against authorized evidence.

Q4 – STUDENT B: SUMMARY CASTING AND RESULT VALIDATION

Student B creates the cast or summary output from long-format scan observations. The objective is to produce a concise view, such as open ports per host, and then prove that the summary totals agree with the source observations.

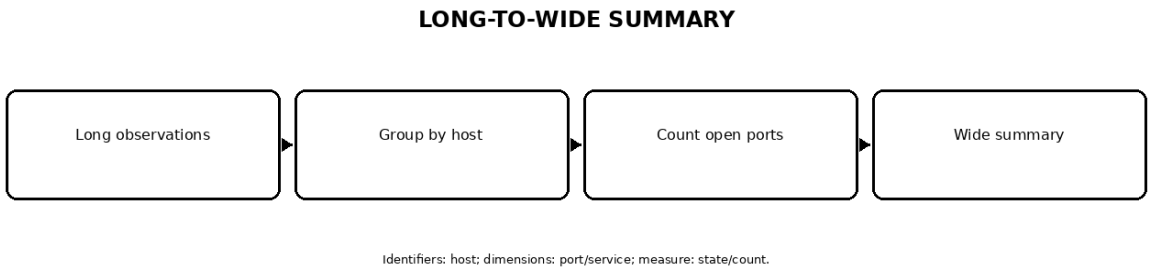


Figure 7. Long observations are grouped and cast into a compact summary.

Input long-format data

```
ip_address,port,protocol,state,service_name
192.168.56.20,22,tcp,open,ssh
192.168.56.20,80,tcp,open,http
```

192.168.56.20,443,tcp,closed,https

192.168.56.30,5555,tcp,open,adb

Identifier, dimensions and measure

Element	Field	Explanation
Identifier	ip_address	Identifies the host being summarized.
Dimensions	port, protocol, service_name	Describe the endpoint categories.
Measured variable	state / open_count	State is categorical; open_count is an aggregated count.

The summary must retain enough detail to answer the intended management or technical question. A host-by-port matrix is useful for comparing known ports, while a per-host open-port count is useful for prioritizing review.

Q4 – OPEN-PORTS-PER-HOST CAST

Host	22/tcp	80/tcp	443/tcp	5555/tcp	Open total
192.168.56.20	1	1	0	0	2
192.168.56.30	0	0	0	1	1
192.168.56.40	0	0	1	0	1

Create per-host open-port totals

```
awk -F, 'NR>1 && $4=="open" {count[$1]++} END {for (h in count) print h "," count[h]}' long_scan.csv | sort
```

Create host-by-service count

```
awk -F, 'NR>1 && $4=="open" {key=$1 "," $5; count[key]++} END {for(k in count) print k "," count[k]}' long_scan.csv | sort
```

A cast can be produced using awk associative arrays or a crosstab utility. The output must include a stable header and a defined ordering so that it can be compared across runs.

Q4 – SUMMARY VALIDATION AND ROUND-TRIP CHECK

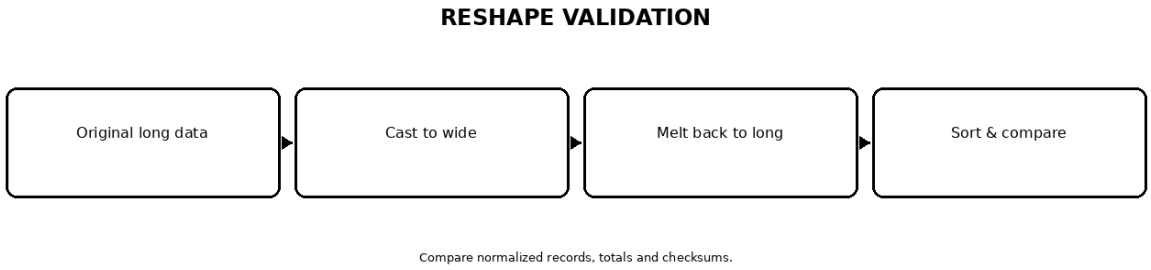


Figure 8. Round-trip validation checks that reshaping has not lost or duplicated observations.

Validation steps

7. Count source long-format rows after excluding the header.
8. Calculate the total number of open observations directly from the long dataset.
9. Sum the per-host open-port counts in the cast output.
10. Confirm the two totals match; investigate any discrepancy.
11. If a wide-to-long round trip is performed, sort both normalized outputs and compare them.

```
awk -F, 'NR>1 && $4=="open" {n++} END{print "Source open observations:",n}' long_scan.csv
awk -F, 'NR>1 {sum += $NF} END{print "Cast total:",sum}' open_ports_per_host.csv
sort long_scan_normalized.csv > long.sorted
sort roundtrip_long.csv > roundtrip.sorted
diff -u long.sorted roundtrip.sorted
```

For checksums, hash normalized data files after sorting and using the same line-ending convention. A matching checksum is strong evidence that the normalized records are identical; it does not prove that the original scan itself was complete or correct.

Q4 – INTERPRETATION OF SUMMARY QUALITY

Why a detailed scan is not always a clearer summary

A full TCP port scan may produce up to 65,535 port observations per host. A full TCP and UDP assessment can create substantially more records, especially when repeated over many hosts and multiple sessions. More data can improve coverage, but it can also increase processing time, storage, noise and the chance that important findings are hidden among low-value observations.

View	Strength	Limitation / best use
Long format	Flexible filtering, grouping and time-based comparison	More rows; useful for analysis and joins
Wide format	Easy side-by-side comparison of selected ports	Many sparse columns; awkward when new ports appear
Per-host summary	Quick prioritization and management overview	Hides individual service details; link to source rows
Per-service summary	Shows prevalence of exposed services	Does not show the full context of each host

The correct output depends on the question. Student B therefore provides both the summary and a link back to the source observations, rather than replacing detailed records with an aggregate.

Q5 – STUDENT B: VERSION–ENUMERATION INTEGRATION

Student B leads integration of OS/Versions and Enumeration findings, including Windows SMB/NetBIOS, Android ADB, Java RMI and NetWare NCP observations. These datasets are primarily host-level observations, so the first join key is the normalized IP address. If a device has a stable device identifier, it may be retained as an additional key, but IP remains the shared inventory link in this assignment.

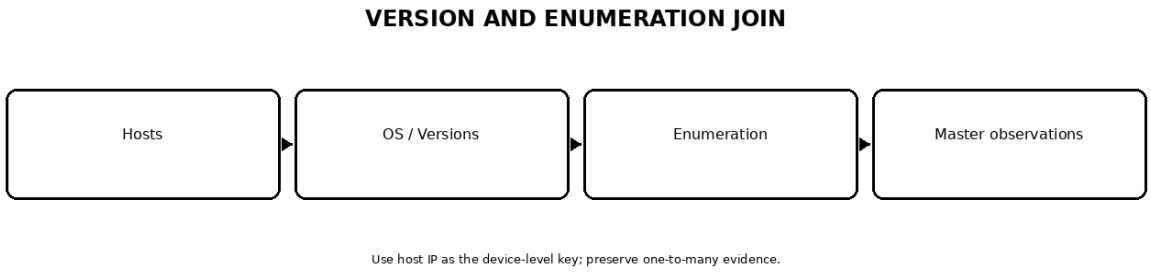


Figure 9. Host-level platform observations are integrated into the master observation table.

Source dataset	Join key	Relationship	Master-table treatment
Hosts	ip_address	One host to many observations	Retain host identity and scope state
OS_Versions	ip_address + scan_id	One host to one or more OS guesses per scan	Keep confidence, method and observation ID
Enumeration	ip_address + scan_id + source_tool	One host to many findings	Keep each finding as a separate row or child table
Services	ip_address + port + protocol + scan_id	One host to many endpoints	Keep service-level granularity

A single flat table can multiply rows if several services and several enumeration findings exist for the same host. A normalized master model can instead maintain one host table plus linked child tables, or use a flattened reporting view with clearly defined row grain.

Q5 – WINDOWS, ANDROID, JAVA AND NETWARE EXCEPTIONS

Platform	Example evidence	Integration rule	Exception to record
Windows SMB / NetBIOS	NetBIOS name, workgroup, SMB response	Join to host by IP and scan_id	Host responds to SMB but no NetBIOS name
Android ADB	Build release, SDK, model from authorized ADB	Join by device IP / registered device ID and scan_id	ADB unavailable or device disconnected
Java RMI	Registry endpoint and bound service names	Join by IP + port + protocol + scan_id	RMI service found but host inventory missing
NetWare NCP	Server identity / NCP service information	Join by IP and scan_id	Tool unavailable or no NCP response
Symbian-type Bluetooth	Service UUID and Bluetooth device	Use device_id; map to host only if	No reliable IP-to-device relationship

	address	approved mapping exists	

Not every enumeration record can be linked to an IP host. Bluetooth service discovery may identify a device address without a corresponding IP address. Such records must remain in a device-observation table and must not be force-joined to an unrelated host.

Exception categories

- ORPHAN_OS: OS/version record has no matching host.
- ORPHAN_ENUM: enumeration finding has no matching host.
- MULTIPLE_OS_GUESS: more than one OS observation exists for a host and scan session.
- DEVICE_ONLY: enumeration source identifies a non-IP device.
- SOURCE_UNAVAILABLE: the platform-specific tool could not run or returned no data.

Q5 – JOIN TYPES AND RECORD-COUNT DIFFERENCES

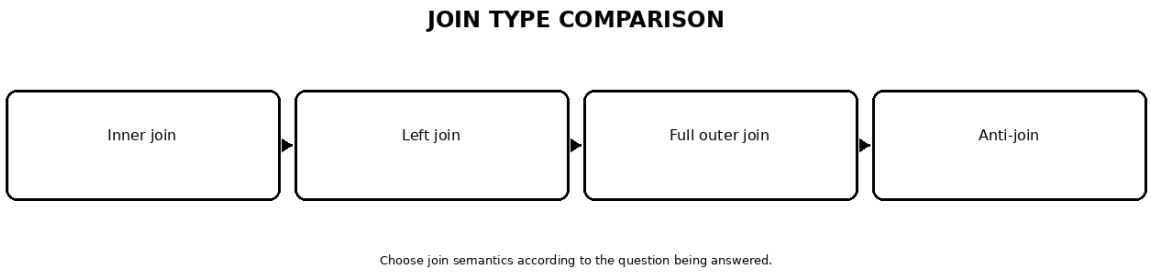


Figure 10. Join selection changes which unmatched records remain in the result.

Join type	What it returns	Use in this pipeline	Risk / interpretation
Inner join	Only keys present in both inputs	Build records with confirmed host and observation match	Drops unmatched evidence from main result; retain anti-join report
Left join	All left-side hosts plus matching observations	Keep every host even if no OS/enumeration result exists	Right-side columns are null for missing matches
Full outer join	All keys from both datasets	Reconciliation and exception analysis	May contain unmatched records requiring review
Anti-join	Rows in one side with no match on the other	Find orphan OS/enumeration findings	Not a master dataset; an exception report

Why record counts differ

Inner joins reduce the result to matching keys. Left joins preserve every left-side row, even when no right-side record exists. Full outer joins preserve unmatched rows from both sides. If a key appears more than once on both sides, a many-to-many join can multiply rows: two OS observations and three enumeration rows for one IP can produce six combined rows if joined without controlling the row grain.

Detect unmatched rows

```
join -t, -v 1 <(sort -t, -k1,1 os_versions.csv) <(sort -t, -k1,1 hosts.csv) > orphan_os.csv
comm -23 <(cut -d, -f1 enumeration.csv | sort -u) <(cut -d, -f1 hosts.csv | sort -u) > orphan_enum_ips.txt
```

In a robust pipeline, use a CSV-aware join tool or load the files into a database because commas, quoted values and headers can break simple shell joins.

Q5 – MASTER TABLE VERIFICATION AND EXCEPTION LOG

Validation procedure

12. Record input row counts for Hosts, OS_Versions and Enumeration.
13. Normalize IP formatting and sort on the selected key.
14. Check duplicate keys and decide whether duplicates are valid repeated observations.
15. Run the selected join and record output row count.
16. Run anti-joins in both directions and preserve unmatched rows.
17. Check that no unexpected row multiplication occurred.

Check	Command / method	Expected result
Input count	wc -l file.csv	Baseline recorded for each dataset
Duplicate keys	sort keyfile | uniq -d	Duplicates explained or flagged
Unmatched records	join -v / comm -23	Exception files created
Output count	wc -l master.csv	Matches expected row grain
Key integrity	sort + unique key check	No accidental duplicate master key

Student B verifies that every OS and enumeration observation is either linked to a known host or explicitly retained as an exception. An exception is not silently deleted merely because the join cannot find a matching row.

Cross-understanding answer

Different join types produce different counts because they preserve different sets of keys. Inner join returns intersections, left join preserves all left records, and full outer join preserves records from both sides. Unmatched hosts or services are detected using anti-joins such as join -v or set differences such as comm -23. Duplicate keys are detected with sort and uniq -d. The evidence consists of input/output counts, unmatched-record files, duplicate reports and a documented expected row grain.

Q6 – STUDENT B: RESULT EXPORT, VERSIONING AND READ-BACK VALIDATION

Student B implements export of cleaned, reshaped and master scan datasets. The naming strategy must preserve earlier sessions and identify the subnet, scan type, timestamp and version. Every output is accompanied by a checksum manifest and a record-count summary.

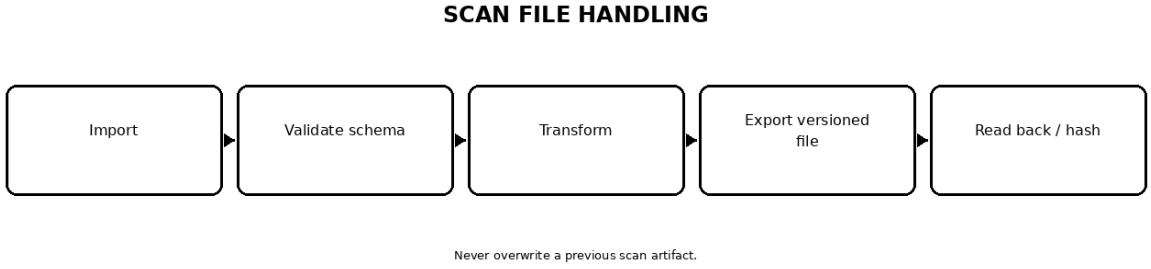


Figure 11. File handling sequence from import through validation and versioned export.

Naming convention

```
subnet_scantype_YYYYMMDD-HHMMSS_vN.csv
192.168.56.0-24_services_20261104-103000_v1.csv
192.168.56.0-24_master_20261104-103000_v1.csv
```

Folder structure

```
assessment-2026-11-04/
  raw/          # original Nmap XML / grepable outputs
  imported/     # schema-checked CSV files
  processed/    # cleaned, reshaped and merged datasets
  reports/      # summaries and finding reports
  manifests/    # checksums, counts, tool versions
```

Raw scan files are read-only after collection. Processed outputs are written to a new versioned filename. A rerun should create a new version or stop if the target file already exists.

Q6 – SAFE EXPORT AND VERSION CONTROL

Prevent accidental overwrite

```
set -o noclobber
outfile="192.168.56.0-24_master_20261104-103000_v1.csv"
if [[ -e "$outfile" ]]; then echo "Refusing to overwrite: $outfile" >&2; exit 1; fi
umask 077
```

```
awk -F, 'BEGIN{OFS=","} {print}' master_scan.csv > "$outfile"
```

The no-clobber setting is a useful guard, but the script should still test for file existence and use a controlled temporary-file-and-rename strategy. Output permissions should restrict access to authorized assessment personnel.

Create checksum and row-count manifest

```
sha256sum "$outfile" > "${outfile}.sha256"
wc -l "$outfile" > "${outfile}.rows"
date -u +%Y-%m-%dT%H:%M:%SZ > "${outfile}.created_utc"
```

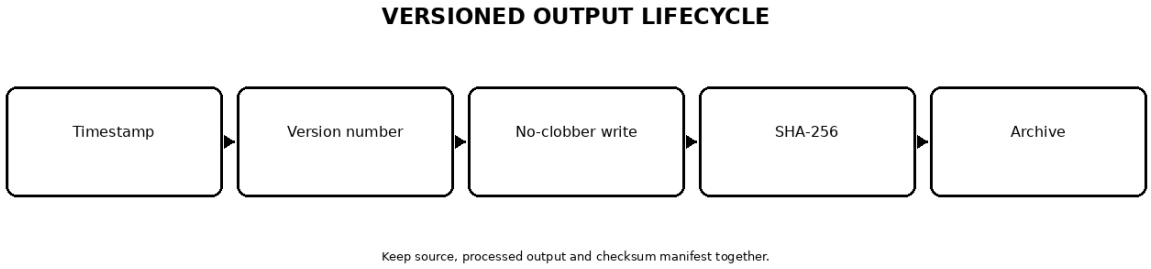


Figure 12. Each exported version is uniquely named, protected from overwrite and accompanied by a checksum.

Q6 – READ-BACK VALIDATION

Re-import and compare

```
sha256sum -c "${outfile}.sha256"
cmp -s "$outfile" reimported_master.csv || { echo "Read-back mismatch" >&2; exit 1; }
diff -u "$outfile" reimported_master.csv
wc -l "$outfile" reimported_master.csv
```

A byte-for-byte comparison is appropriate when the re-import/export process preserves the exact file representation. If the import process changes quoting, line endings or column order, compare normalized records instead and document the normalization method.

Student B validation record

Validation item	Evidence to retain	Pass condition
Output naming	Filename and scan-session ID	Unique, descriptive and timestamped
No overwrite	Existing-file test log	Previous version remains unchanged
Checksum	SHA-256 manifest	sha256sum -c succeeds
Read-back	cmp / diff output	No unexplained differences
Record count	wc -l before and after	Counts match expected headers and rows
Permissions	ls -l output directory	Only authorized group can read/write

A checksum proves that a file has not changed since the checksum was created. It does not prove that the scan data is accurate or that the correct scope was scanned; those claims require separate validation evidence.

Q7 – STUDENT B: REUSABLE FUNCTION LIBRARY DESIGN

The reusable Bash library is named scanlib.sh and is loaded using source. Student B leads reshaping, merging and file-handling functions. Each function accepts explicit input paths, supports sensible defaults, returns a status code and writes diagnostics to standard error.

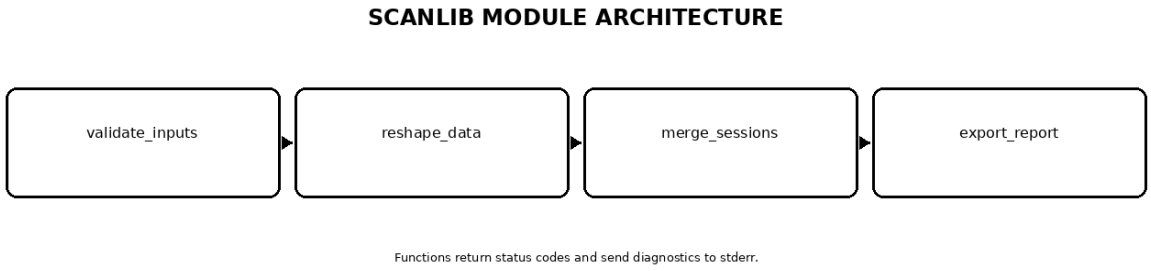


Figure 13. Student B function responsibilities within the reusable scan library.

Student B function proposal

Function	Inputs	Output	Return code
reshape_wide_to_long	wide CSV, output CSV	Long-format observations	0 success; non-zero failure
cast_host_summary	long CSV, output CSV	Per-host open-port summary	0 success; non-zero failure
merge_observations	hosts, OS, enumeration, output	Integrated master / exception files	0 success; non-zero failure
export_versioned	input, output directory, prefix	Versioned output + manifest	0 success; non-zero failure
verify_export	output file, checksum, reference	Validation log	0 match; non-zero mismatch

Design principles

- Validate arguments before opening files.
- Use temporary files and only publish final outputs after successful completion.
- Do not overwrite source scan files.
- Write diagnostics to stderr so stdout can be piped into another operation.
- Return explicit exit codes so the calling pipeline can stop on failure.

Q7 – STUDENT B: FUNCTION INTERFACES AND ERROR HANDLING

Example function interfaces

```
reshape_wide_to_long() {
  local input="${1:-wide_scan.csv}"
  local output="${2:-long_scan.csv}"
  [[ -r "$input" ]] || { echo "Missing input: $input" >&2; return 2; }
  # Transformation body is implemented and tested in the lab.
}
merge_observations() {
  local hosts="${1:-hosts.csv}"
  local osfile="${2:-os_versions.csv}"
  local enumfile="${3:-enumeration.csv}"
  local output="${4:-master_observations.csv}"
  for f in "$hosts" "$osfile" "$enumfile"; do
    [[ -r "$f" ]] || { echo "Cannot read: $f" >&2; return 2; }
  done
  # Join logic must preserve unmatched observations.
}
```

Error-handling shell options

```
set -euo pipefail
trap 'echo "Pipeline failed at line $LINENO" >&2' ERR
```

set -euo pipefail is useful but does not eliminate the need to check every command's assumptions. Pipelines involving grep may return status 1 when no match exists; the library must handle expected "no match" results deliberately rather than treating them as unexpected failures.

Q7 – STUDENT B: INTEGRATION, COUNTS AND REPRODUCIBILITY

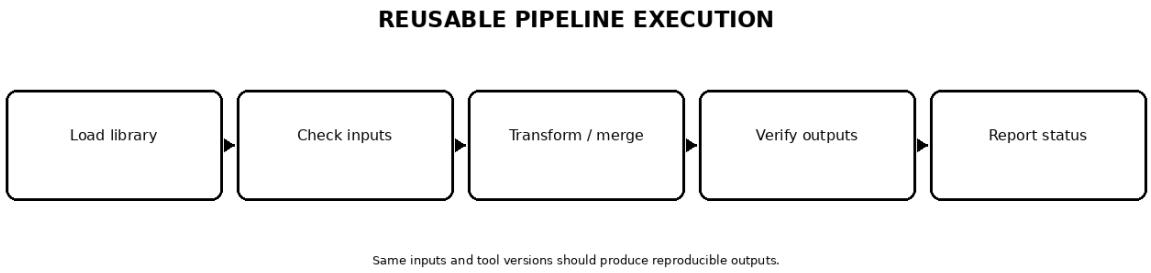


Figure 14. Reusable pipeline execution and validation path.

Integration sequence

18. Read validated input paths and confirm the scan session identifier.
19. Run the reshape function and record the long-format row count.
20. Run the cast function and compare summary totals with source observations.
21. Merge OS/version and enumeration records with host identity; create exception outputs.
22. Export master and summary outputs using unique versioned names.
23. Run read-back validation and compute checksums.

Stage	Example record count	Evidence
Long-format observations	250	long_scan.rows
Host summary rows	50	host_summary.rows
Matched OS observations	42	os_matched.csv
Unmatched OS observations	3	os_unmatched.csv
Enumeration observations	65	enumeration.csv
Final master observations	107	master_scan.rows

Counts are illustrative. In the actual practical run, counts must be measured from the created files. A reproducibility test reruns the pipeline with the same inputs, configuration and tool versions and compares output hashes.

Q7 – STUDENT B: EXPOSURE REPORTING AND MANAGEMENT SUMMARY

Student B prepares an exposure or service-version risk report using service/version observations. The report should translate technical scan data into a concise view for a university manager: which assets need attention, why they matter and what action is recommended.

Report fields

Field	Purpose
Asset / host	Identifies the approved system
Port / protocol	Shows the exposed endpoint
Service / product / version	Identifies the observed technology
Evidence source	Nmap output, enumeration result or lab evidence ID
Risk indicator	Outdated version, unexpected exposure or unsupported platform
Confidence	Separates confirmed evidence from uncertain fingerprinting
Recommended action	Patch, restrict, disable, upgrade or investigate
Owner / status	Assigns remediation responsibility

Illustrative management summary

Priority	Count (example only)	Management action
High	2	Confirm owner and prioritize remediation review
Medium	5	Schedule configuration or version review
Low / informational	8	Track in normal maintenance cycle
Needs verification	3	Collect additional evidence before risk rating

The counts above are example placeholders, not actual vulnerability findings. The report should never label a version vulnerable solely because it looks old; validate the product/version against a trusted advisory or approved vulnerability source and record the evidence date.

Management requirement addressed

The report helps management decide where to allocate remediation resources. It groups findings by asset and priority, retains traceability to the original scan evidence and distinguishes confirmed risks from items requiring verification.

Q7 – STUDENT B: BOTTLENECK ANALYSIS AND OPTIMIZATION

Student B analyzes pipeline measurements, identifies a bottleneck and implements one refinement. A common inefficient pattern is invoking grep once for every input row. Process startup overhead becomes significant as the number of records grows. A single awk pass can perform the same lookup in memory.

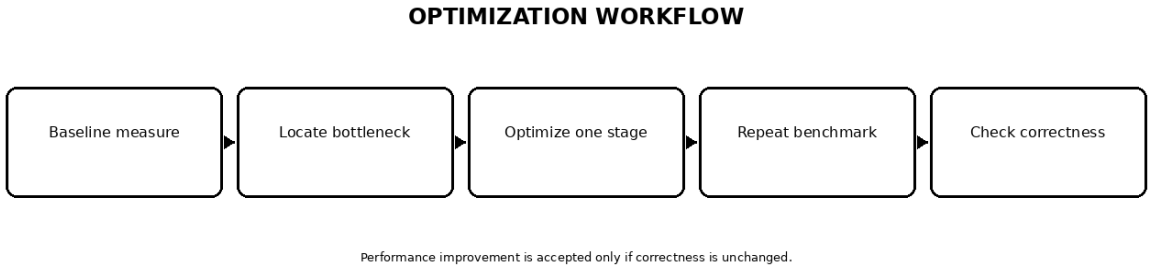


Figure 15. Optimization must be benchmarked and revalidated.

Baseline versus optimized approach

```
# Baseline pattern: repeated process creation (illustrative)
while IFS=, read -r ip port proto service; do
  grep -F "$ip" hosts.csv >> matches.csv
done < services.csv
# Optimized pattern: load host keys once and scan service rows once
awk -F, 'FNR==NR {known[$1]=1; next} FNR>1 && ($1 in known) {print}' hosts.csv services.csv
```

Metric	Baseline	Optimized	Interpretation
Input records	5,000	5,000	Same input set
Process launches	Thousands	One awk process	Lower process overhead
Elapsed time	Measure with time	Measure with time	Use repeated runs and median
Correctness	Compare output rows	Compare output rows	Must be identical after normalization

No performance numbers are invented here. Record real elapsed time using time or /usr/bin/time -v on the same machine, with the same input and environment. Compare at least three runs and report the median.

Optimization acceptance criteria

- Output records and keys match the baseline after normalization.
- No unmatched rows disappear.
- Memory use remains acceptable for the target dataset.
- The optimized code remains understandable and maintainable.

Q7 – STUDENT B: MULTI-SESSION WORKFLOW AND CHANGE VALIDATION

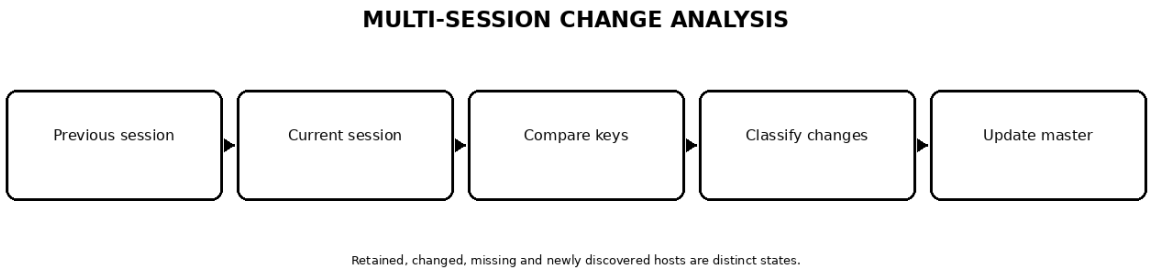


Figure 16. Multi-session comparison classifies host changes before the master inventory is updated.

Student B modifies the workflow to compare a previous scan session with a current session. The comparison must identify retained hosts, updated observations, hosts no longer observed and newly discovered hosts. A host missing from the latest scan is not automatically declared removed; it may have been offline or filtered.

Change class	Definition	Pipeline action
Retained	Host appears in both sessions with no material change	Keep host; update last_seen
Updated	Host appears in both sessions but service / OS / enumeration changed	Store new observation and compare history
Unmatched / not observed	Host was previously present but not found now	Flag for verification; do not delete automatically
Newly discovered	Host appears only in current session	Check scope and owner before adding to master
Out of scope	New observation is outside approved range	Quarantine and escalate

Comparison commands

```
ndiff previous_scan.xml current_scan.xml > scan_delta.txt
comm -23 previous_host_ips.txt current_host_ips.txt > not_seen_current.txt
comm -13 previous_host_ips.txt current_host_ips.txt > newly_seen_current.txt
```

Both host lists must be normalized and sorted before comm is used. ndiff can compare Nmap XML scan results, while comm provides simple set differences for host identifiers.

Q7 – MULTI-SESSION TEST MATRIX AND TRADE-OFFS

Test	Previous session	Current session	Expected classification
M-01	192.168.56.20 present	192.168.56.20 present, same services	Retained
M-02	192.168.56.30 Android 13	192.168.56.30 Android 14	Updated
M-03	192.168.56.40 present	Not observed	Unmatched / verify
M-04	Not present	192.168.56.50 appears	Newly discovered; scope check
M-05	Not present	192.168.57.10 appears	Out of scope; quarantine

Evidence to retain

- Previous and current raw scan files, timestamps and scan options.
- Normalized host lists and the ndiff / comm comparison outputs.
- A change register with classification, reviewer and decision.
- Checksums for both source sessions and the resulting delta file.

Trade-offs

Repeated scans improve change visibility but increase traffic, storage and the chance of inconsistent observations due to transient network conditions. A host may appear missing because it was powered off or filtered. A new host may be a legitimate device or an out-of-scope system. The workflow therefore flags changes for owner review instead of automatically deleting or approving them.

Comparison with the original single-session pipeline

The original pipeline processes one scan session at a time and creates a point-in-time dataset. The revised pipeline adds a session identifier, historical observation storage, normalized host keys and a change-classification stage. This supports trend analysis but requires retention rules, stable schemas and more storage.

Q7 – JOINT EVALUATION: PIPELINE DESIGN RATIONALE

The shared evaluation is included because the source assignment explicitly requires both students to understand the integrated pipeline. This section does not provide Student A's individual implementation; it explains the system-level rationale needed to defend Student B's integration work.

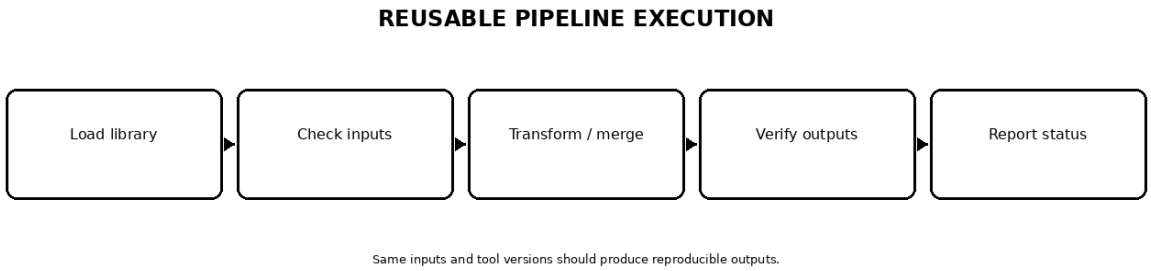


Figure 17. Structured pipeline sequence.

Why the stages must be ordered

Stage	Purpose	Failure if skipped
Import	Read source outputs and preserve provenance	Untraceable or incomplete inputs
Validation	Check schema, keys, ranges and scope	Malformed data contaminates later stages
Cleaning	Normalize safe variations and flag defects	Duplicates and inconsistent labels distort counts
Reshaping	Convert between analysis-friendly structures	Summaries become difficult or inconsistent
Merging	Connect hosts, services, OS and enumeration	Observations remain fragmented or are incorrectly linked
Export	Write versioned outputs and manifests	Previous evidence may be overwritten or results cannot be reproduced

Benefits of modular functions

- Correctness: each function has a narrow responsibility and testable contract.
- Maintainability: a change to service normalization does not require rewriting export logic.
- Efficiency: repeated operations can be optimized independently.
- Reproducibility: function inputs, versions and outputs can be logged.

Costs and limitations

- Validation and exception handling require additional development and execution time.
- Repeated scans create more storage and require retention and access-control policies.
- Different scanners may use inconsistent naming, formats and confidence models.
- Schema changes can break older scripts unless versioning and migration rules are maintained.

PART B – FINAL CONCLUSION AND SUBMISSION CHECKLIST

Final conclusion

The Student B work establishes a reliable path from raw scan observations to validated, standardized, integrated and reportable security-assessment data. The service, OS/version and enumeration schemas preserve host relationships and source provenance. Validation checks required fields, address and port ranges, protocol values, service-name consistency and approved scope. Cleaning operations retain review and rejected records so that no finding disappears without explanation.

The summary-casting process creates useful per-host or per-service views and validates totals against the source long-format observations. The OS/version and enumeration integration compares join types, detects orphan records and avoids accidental row multiplication. The file-handling pipeline uses versioned names, no-clobber protection,

checksums and read-back comparisons. Finally, the reusable scan library adds modular transformation, merge, export and verification functions, while multi-session comparison distinguishes retained, updated, missing and newly discovered hosts.

Student B evidence checklist

- Services, OS/Versions and Enumeration CSV headers and sample records are included.
- Relationship keys and platform-specific enumeration mapping are documented.
- At least four enquiry questions have a command and acceptance criterion.
- Validation and service-name normalization rules are documented.
- Review, rejected and duplicate records are preserved with reasons.
- Host summary/cast output is reconciled against source observations.
- OS/version and enumeration joins include unmatched-record analysis.
- Export naming, checksums, read-back and row-count evidence are retained.
- Student B function proposals, integration tests and reproducibility checks are retained.
- Management summary, benchmark comparison and multi-session change tests are completed with actual lab results.

Important: sample datasets, counts and timelines in this document are illustrative. Replace them with actual outputs from the pair's authorized lab, preserve raw evidence, and complete the contribution record separately according to the team's real work.